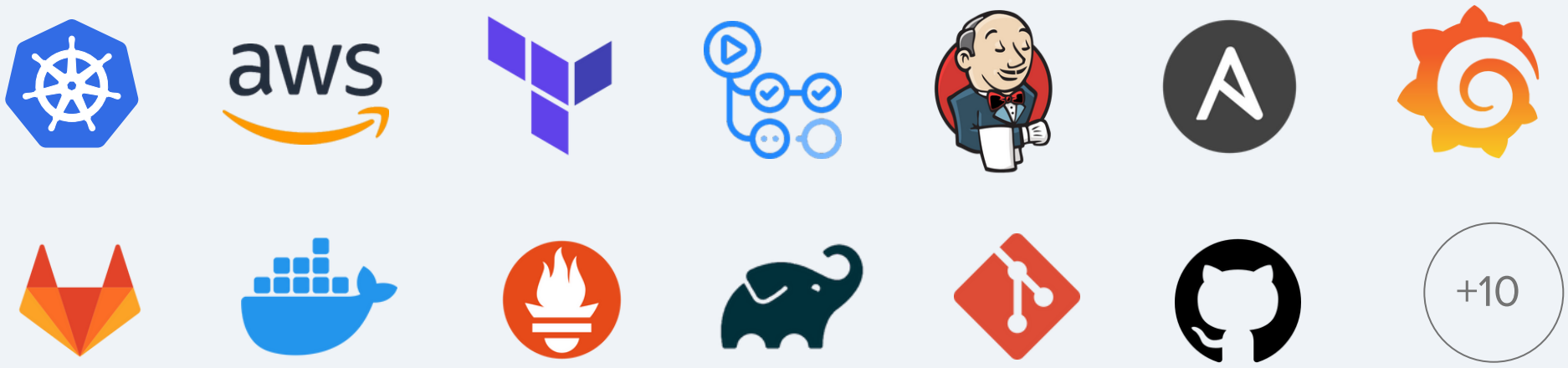
14-Day DevOps Interview Prep Roadmap



newsletter.devopscube.com

This roadmap is designed to help you cover all the important areas from basic concepts like Linux, Networking, and Git, to hands-on practice, cloud system design, kubernetes and interview techniques.

Here's what's included:

- A detailed list of DevOps topics you should know
- Tips on how to plan and research companies
- Guidance for behavioral interview questions
- Resume preparation tips and useful tools

Week 1: Foundations & Core Technologies

🗓️ Day 1: Strategic Planning & Company Research

🎯 **Goal:** Understand the company's context, the specific role, and the likely interview focus areas.

🛠️ Activities:

- Research the company: Products, services, scale, business drivers, primary infrastructure (**AWS, Azure, GCP, On-prem?**).
- Deep-dive into the Job Description: Identify key responsibilities, required tools (**Terraform, Ansible, Jenkins, Docker, Kubernetes,** etc.), and experience level.
- Investigate their Tech Stack: Check tech blogs, conference talks (YouTube), open-source projects, and employee LinkedIn profiles.
- Identify Company Values / Engineering Principles.
- Consult Interview Resources: **Glassdoor, Blind, Reddit** (r/devops), for company-specific DevOps interview insights. Note down common question types (System Design, Coding, Troubleshooting, Behavioral).
- Start Your Questions List: Begin compiling insightful questions to ask your interviewers.

✅ **Output:** A clear picture of the company/role context, a prioritized list of study topics/tools, understanding of the likely interview format, and initial questions for interviewers.

🗓️ Day 2: 🐧 Linux & Foundational Concepts (OS/Networking)

🎯 **Goal:** Make sure you're strong in basic Linux commands and understand the key concepts of operating systems and networking.

🛠️ **Activities:**

Mastering Linux Commands

- File/Directory Management: find, locate.
- Text Processing: less, more, head, tail, grep, sed, awk.
- Permissions: chmod, chown, sudo; understand user/group/other permissions (symbolic & octal).
- Process Management: ps, top, htop, kill, nice, systemctl/service.
- Archiving/Compression: tar, gzip, gunzip.
- Disk Usage: df, du.
- Basic Networking Tools: ping, ip addr (or ifconfig), netstat, ss, traceroute, dig, nslookup, curl, wget, lsof.
- Process Management: ps, top, htop, kill, nice, systemctl, service, uptime, dmesg | tail, vmstat 1, mpstat -P ALL 1, pidstat 1, iostat -xz 1, free -m, sar -n DEV 1, sar -n TCP,ETCP 1

Operating System Concepts:

- Filesystem Hierarchy Standard (FHS).
- /proc filesystem
- Kernel vs. Shell interaction.
- Kernel Space and User Space
- How Linux handles Interrupts
- I/O Redirection & Piping (**>, >>, |, 2>**).
- Environment Variables (**export, .bashrc/.zshrc**).
- Processes vs. Threads.
- Virtual Memory concepts.
- Inodes and File Descriptors (including standard streams 0, 1, 2).
- Linux Boot Process (high-level: BIOS/UEFI -> Bootloader -> Kernel -> Init System).
- Init Systems (**systemd** vs **SysVinit** concepts).
- Load Average

Networking Fundamentals:

- OSI Model: Understand the 7 layers and the primary function of each (esp. Layers 1-4, 7).
- TCP/IP Model: Relationship to OSI; focus on key protocols at each layer.
- TCP vs. UDP: Key differences, use cases, 3-way handshake (TCP).
- IP Addressing: Basics of IPv4 addresses, subnet masks, private vs. public IPs, CIDR notation (briefly).
- DNS: Resolution process (recursive vs. iterative), common record types (**A, AAAA, CNAME, MX, TXT, NS**).
- HTTP/HTTPS: Request methods (**GET, POST**, etc.), status codes (categories 2xx-5xx), basics of HTTPS/TLS handshake and certificates.
- Common Ports (e.g., 22, 53, 80, 443).
- Concepts: Default Gateway, Routing basics.

✅ Output: Confidence using the Linux command line for common tasks; foundational understanding of OS structure, core networking principles, and the OSI model.

🗓️ Day 3: DevOps Concepts, Cloud Fundamentals & Git Essentials

🎯 **Goal:** Learn the key ideas of DevOps, get familiar with the basics of cloud platforms, and understand how to use Git for version control.

🛠️ Activities:

- Review DevOps Pillars: CI/CD principles, Infrastructure as Code (IaC) importance, Monitoring vs. Observability, Configuration Management goals, **DevOps Culture** (collaboration, feedback loops, automation).

- Study Cloud Fundamentals: (Focus on their primary provider: AWS, Azure**,** or GCP): Core Compute, Storage, Networking (VPC/VNet, SG/NSG, LB), IAM, Managed Databases (basic concepts).

- **Version Control with Git**:
    - Core Concepts: Understand Repository, Working Directory, Staging Area (Index), Commit, Branch, Merge (including different strategies like fast-forward vs recursive), Remote (e.g., GitHub, GitLab, Bitbucket), HEAD, Tags.
    - Essential Commands: Practice git init, git clone, git status, git add, git commit -m, git log (and variations like --oneline, --graph), git diff, git branch, git checkout/git switch, git merge.
    - Working with Remotes: Practice git remote add, git fetch, git pull (and --rebase), git push.

- Branching Strategies: Understand the concepts behind Feature Branching, Gitflow (high-level), Trunk-Based Development (high-level). Why use branches?

- Collaboration: Basic understanding of how to handle Merge Conflicts.

- Ignoring Files: Understand and use **.gitignore.**

💡 **Interview Tips**

- **Highlight automation:** pre-commit hooks, automated testing on PRs, branch protection rules, and merge policies that enforce code quality

- **Highlight automation:** pre-commit hooks, automated testing on PRs, branch protection rules, and merge policies that enforce code quality

- **Address collaboration:** conflict resolution strategies, code review processes, fork vs branch workflows, and maintaining clean history

- **Cover advanced concepts:** git submodules for multi-repo projects, git hooks for automation, cherry-picking for selective fixes, and disaster recovery with reflog

✅ **Output:** Strong grasp of DevOps principles, foundational cloud knowledge, and confidence using Git for version control tasks including branching, merging, and remote collaboration.

🗓️ Day 4: Scripting for Automation (Python/Bash)

🎯 **Goal:** Develop practical scripting skills, leveraging Linux fundamentals and assuming code is managed in Git, for automating common DevOps tasks.

🛠️ **Activities:**

- Select primary language: **Python** (highly recommended) and/or Bash (essential).
- Language Basics: Data types, control flow (loops, conditionals), functions, error handling (**try-except** in Python).
- Practice Scripting Tasks (incorporating Linux commands):
    - File I/O (reading configs, writing logs/reports).
    - API Interaction (using requests in Python or curl/jq in Bash).
    - Data Parsing (JSON using json lib or jq, YAML using PyYAML).
    - OS Interaction (running commands via subprocess or os.system, managing processes, gathering system info).
    - Combining tools using pipes (|) within scripts.
- Focus on writing clean and easy-to-read code. Make sure it handles errors properly, uses functions for better structure, and can be run multiple times without causing issues (idempotency). Also, keep your code tracked in Git.

✅ **Output:** Ability to write simple-to-intermediate scripts for common automation needs; referenceable code snippets leveraging OS commands, managed with Git.

🗓️ Day 5: Infrastructure as Code (IaC) - Part 1 (Fundamentals & Tooling)

🎯 **Goal:** Understand IaC principles and gain hands-on experience with a primary tool (e.g., Terraform), managing code with Git.

🛠️ **Activities:**

- Explore IaC Concepts: Declarative approach, State Management, Idempotency, Modularity. (Code should be versioned in Git).
- Setup Chosen Tool: Install Terraform (or CloudFormation/ARM/Pulumi).
- Work Through Basics: Define simple cloud resources (VM, Storage Bucket, Security Group, basic network).
- Understand Core Workflow: terraform init, terraform plan, terraform apply, terraform destroy, terraform fmt, terraform validate.
- Learn about: Input Variables, Resource Outputs, Basic Dependencies, Data Sources.
- Understand lifecycle blocks, prevent_destroy, taint, and using -replace
- Know how workspaces work and when to use them vs separate modules or stacks. Example use cases: dev/staging/prod isolation with distinct states.

✅ **Output:** Understanding of IaC core concepts and basic proficiency defining/managing simple infrastructure with your chosen tool, using Git for code management.

🗓️ Day 6: Infrastructure as Code (IaC) - Part 2 (Intermediate Concepts)

🎯 **Goal:** Explore more advanced IaC features, best practices, and structure, managed within Git.

🛠️ **Activities:**

- Learn Modules: Use Terraform Modules (local and remote/registry) or (CloudFormation Nested Stacks) for reusable infrastructure components. (Modules versioned in Git).
- Learn About Statelock: Statelock backends (e.g., S3, Azure Blob Storage), State Locking (critical for teams), terraform workspace.
- Handle Sensitive Data: Integrate with secrets management tools (Vault, AWS Secrets Manager, Azure Key Vault), avoid hardcoding secrets. Use sensitive variable flags.
- Build More Complex Setups: E.g., Provision a web server cluster behind a load balancer with database connectivity.
- Explore IaC Best Practices**:** Linting (tflint), static analysis (tfsec/checkov), directory structure patterns, version constraints.
- Understand cost optimization, performance tuning, and multi-region deployments

- Prepare for questions like:
  - Migrating infrastructure to Terraform.
  - Splitting monolithic configs into modules.
  - Handling hundreds of environments/projects.
  - Troubleshooting resource/resource drift or state corruption

💡 **Interview Tips**

- **Mention pros & cons:** e.g., workspaces reduce duplication but can make backend tricky; folder structure keeps clear separation
- **Highlight automation:** PR checks, plan in CI, manual apply gates, environment promotion.
- **Include tooling:** Atlantis, Terragrunt, Spacelift help orchestrate multi-env deployments.
- **Emphasize best practices:** isolated state, shared modules, promotion pipeline, and environment-specific variable management.

✅ **Output:** Ability to structure IaC code effectively using modules, understand state management trade-offs, handle secrets securely, and apply best practices, all managed via Git.

🗓️ Day 7: Configuration Management (Ansible/Chef/Puppet)

🎯 **Goal:** Understand the role of configuration management and gain basic proficiency in one tool (e.g., Ansible), managing code with Git.

🛠️ Activities:

- Differentiate: IaC (Provisioning) vs. Configuration Management (Configuration & Maintenance). Understand when to use which. (Code/Playbooks versioned in Git).
- Understand Key Concepts: **Idempotency, Push vs. Pull** models, **Agent vs. Agentless.**
- Get Hands-on (Ansible recommended for simplicity):
    - Write basic **Playbooks**. Use common modules (apt/yum, copy, template, service, user, file).
    - Define **Tasks** ensuring idempotency.
    - Understand **Inventory** (static and dynamic).
    - Explore basic **Roles** structure for organization and reusability.
    - Use Handlers and Variables.
    - Look into ansible-vault for encrypting sensitive data.

✅ **Output:** Understanding of configuration management principles like idempotency and the ability to automate basic server setup/configuration tasks idempotently, using Git for code management.

Week 2: Orchestration, Pipelines, Operations & Practice

devopscube.com

🗓️ Day 8: Containers (Docker)🐳

🎯 **Goal:** Master Docker fundamentals for building, pushing, and running applications consistently. (Dockerfiles managed in Git).

🛠️ Activities:

- Review Core Concepts: Images vs. Containers, Layered Filesystem, Namespaces & Cgroups (high-level), Benefits (Consistency, Isolation, Density).
- Practice docker CLI: build, run, ps, stop, rm, images, rmi, logs, exec, inspect, network, volume.
- Write Effective Dockerfiles: Understand instructions (FROM, RUN, COPY, ADD, WORKDIR, EXPOSE, USER, CMD vs ENTRYPOINT), **Multi-Stage Builds**, minimizing layers, security best practices (non-root user).
- Understand Docker Networking: Port mapping, bridge/host/overlay network drivers.
- Learn Docker Volumes & Bind Mounts: Managing persistent data.
- Explore docker-compose: Define and run multi-container applications locally for development environments. (docker-compose.yml versioned in Git).

✅ **Output:** Solid understanding of Docker, ability to write efficient and secure Dockerfiles, manage container lifecycles/data/networking, and use docker-compose.

🗓️ Day 9: Container Orchestration (Kubernetes/ECS)☸️

🎯 **Goal:** Understand the need for orchestration and learn the fundamentals of a key platform (Kubernetes highly recommended). (YAML manifests managed in Git).

🛠️ Activities:

- Identify Problems Solved: Scaling, Self-Healing, Service Discovery, Load Balancing, Rolling Updates/Rollbacks, Resource Management.

- **Learn Kubernetes Basics**
    - Core Objects: Pod (and containers within), Service (types: ClusterIP, NodePort, LoadBalancer), Deployment (and ReplicaSet), Namespace, ConfigMap, Secret. Understand their purpose and relationships.
    - Architecture Overview: Control Plane (api-server, etcd, scheduler, controller-manager), Worker Nodes (kubelet, kube-proxy, Container Runtime).
    - Practice kubectl: get, describe, apply -f, delete -f, logs, exec, port-forward, config, explain. Learn YAML manifest structure.
    - Deploy & Expose: Run a simple application using a Deployment and expose it via a Service. Interact with Pods.

- Compare (Optional): Briefly understand managed K8s (EKS, AKS, GKE) and alternatives (ECS, Fargate, Nomad).

✅ **Output:** Clear understanding of container orchestration; hands-on familiarity with Kubernetes core resources, architecture, and basic kubectl usage, managing manifests with Git.

🗓️ Day 10: CI/CD Pipelines & Tools

🎯 **Goal:** Design and understand CI/CD workflows and common tooling, triggered from Git repositories.

🛠️ Activities:

- Detail CI/CD Stages: Source (Git commit/merge trigger) -> Build -> Test (Unit, Integration, Static Analysis) -> Artifact Storage -> Deploy (Staging -> Prod) -> Verify/Monitor/Rollback. Understand the difference between CDelivery and CDeployment.

- Explore Pipeline-as-Code: Syntax and structure for Jenkinsfile (Declarative/Scripted), .gitlab-ci.yml, GitHub Actions workflows. (Pipeline definitions stored in Git).

- Practice with One Tool: Define a simple pipeline (trigger, stages, steps, basic build/test execution). Integrate with source control (Git).

- Understand Artifact Management: Using registries like Docker Hub, ECR, ACR, GCR, Nexus, Artifactory. Versioning strategies.

- Study Deployment Strategies: **Rolling Update, Blue/Green Deployment, Canary Release, A/B Testing** (conceptual). Discuss pros, cons, implementation considerations, and rollback mechanisms.

✅ **Output:** Ability to design basic CI/CD pipelines triggered by Git events, understand pipeline-as-code syntax for a major tool, discuss various deployment strategies and trade-offs.

🗓️ Day 11: Monitoring, Logging & Observability📈

🎯 **Goal:** Understand how to monitor systems effectively using the pillars of observability (metrics, logs, traces) and the tools involved.

🛠️ **Activities:**

- Observability Pillars: Metrics, Logging, and Trace.

- **Explore Common Tooling Categories:**
  - Metrics/Alerting: Time-series databases (Prometheus, InfluxDB), Visualization (Grafana), Alerting (Alertmanager), Cloud Provider tools (CloudWatch, Azure Monitor).
  - Logging: Log shippers (Fluentd, Logstash, Filebeat), Storage/Indexing (Elasticsearch/OpenSearch, Loki), Visualization (Kibana, Grafana). Cloud Provider tools (CloudWatch Logs, Azure Log Analytics).
  - Tracing: Standards (OpenTelemetry), Tools (Jaeger, Zipkin, Cloud Provider tools).

- Define Monitoring Strategy: Consider what key metrics/logs/traces to collect for different system types (web app, database, K8s cluster).

- Understand Basic Alerting: Principles of setting meaningful thresholds, reducing noise, routing notifications.

✅ **Output:** Solid understanding of observability concepts (the 'three pillars'), familiarity with the types of tools used in each category, and the ability to discuss basic monitoring strategies.

🗓️ Day 12: Security & Advanced Networking / Infrastructure Concepts

🎯 **Goal:** Integrate fundamental security (DevSecOps) and explore more advanced networking and infrastructure concepts relevant in complex environments.

🛠️ **Activities:**

Security (DevSecOps) Integration:

- Secrets Management: Tools (Vault, AWS Secrets Manager, Azure Key Vault), secure injection patterns.
- IAM Best Practices: Principle of Least Privilege review; Role-Based Access Control (RBAC) in cloud and K8s.
- Vulnerability Scanning: Tools (Trivy, Clair, Snyk, Dependabot, SAST/DAST concepts) and integration points (CI, registry).
- Pipeline Security: Securing build environments, artifact signing/verification.
- Infrastructure Hardening basics (e.g., CIS benchmarks awareness).

Security (DevSecOps) Integration:

- Firewalls: Stateful vs. Stateless concepts.
- Cloud Network Security: Security Groups (AWS SG, Azure NSG) vs. Network ACLs (NACLs). WAF basics.
- Secure Access Patterns: Bastion Hosts / Jump Boxes, VPNs, Zero Trust concepts (high-level).

Advanced Networking & Infrastructure:

- Cloud Networking: VPC Peering / Transit Gateway / VNet Peering, PrivateLink / Private Endpoints, Direct Connect / ExpressRoute concepts.

- Load Balancing: Layer 4 (TCP/UDP) vs. Layer 7 (HTTP/S) load balancers, common algorithms (Round Robin, Least Connections, IP Hash), Health Checks.

- Content Delivery Networks (CDN): Basic concepts (caching, edge locations, invalidation).

- Kubernetes Networking: CNI plugin overview, Ingress controllers (e.g., Nginx Ingress, Traefik), Network Policies for pod-to-pod traffic control.

- Service Mesh Concepts (Optional, if relevant): e.g., Istio, Linkerd (mTLS, traffic management, observability)

✅ **Output:** Awareness of key security practices integrated into DevOps (DevSecOps), foundational understanding of essential cloud/container networking, plus exposure to more advanced networking, security, and infrastructure topics.

🗓️ Day 13: System Design Practice & Troubleshooting

🎯 **Goal:** Apply gained knowledge to design DevOps-related systems and practice common troubleshooting scenarios systematically.

🛠️ Activities:

- Tackle DevOps System Design Prompts:
- Focus on CI/CD pipeline design (consider security, testing, artifacts, multi-environment promotion).
- Architect highly available/scalable infrastructure on a specific cloud (consider load balancing, auto-scaling, databases, state, disaster recovery).
- Design monitoring/logging/alerting strategies for a given architecture (consider dashboards, alert fatigue).
- Consider deployment strategies (Blue/Green, Canary) implementation details and automation.

- Practice Diagramming: Use simple block diagrams to communicate architecture. Focus on explaining components, data flow, trade-offs, scalability, reliability, and cost considerations.
    - Practice applying approaches like USE/RED for identifying bottlenecks or the "Five Whys" for root cause analysis.
    - Work through hypothetical failures in CI/CD, IaC, K8s, Linux services. Define steps: Observe symptoms -> Gather data (logs, metrics, traces) -> Formulate hypothesis -> Test hypothesis -> Implement fix/rollback -> Verify -> Document.
    - Practice using debugging tools learned (kubectl describe/logs, system logs (journalctl), monitoring dashboards, network diagnostic tools (tcpdump basics)).

✅ **Output:** Increased confidence tackling system design questions relevant to DevOps and articulating systematic troubleshooting approaches for various scenarios.

🗓️ Day 14: Behavioral Prep, Final Review & Mock Interviews

🎯 **Goal:** Practice behavioral answers, do final reviews, take mock interviews to handle pressure, and make sure you're fully prepared for the interview day.

🛠️ Activities:

Behavioral Answers:

- Review Company Values/Principles (from Day 1).
- Refine STARR Stories (Situation, Task, Action, Result, Reflection). Ensure relevance to DevOps (collaboration, automation, problem-solving, incident response, learning, leadership). Practice telling them out loud smoothly.
- Practice Your Elevator Pitch (< 1 minute).
- Finalize Your Questions for interviewers - ensure they are thoughtful and specific to the role/team/company.

Final Technical Interview:

- Quickly review notes/mind maps from all previous days – focus on core concepts, commands, patterns, trade-offs.
- Explain key ideas out loud briefly (e.g., "Explain IaC state", "Explain K8s Service types", "Explain TCP 3-way handshake", "Explain Git branching")

Final Technical Interview:

- Arrange sessions covering the breadth of topics: Linux/Fundamentals, Git, Scripting, IaC, Containers/K8s, CI/CD, Monitoring, Networking/Security, System Design, Troubleshooting, Behavioral.

- Get constructive feedback and identify any last-minute areas needing quick reinforcement.

- Logistics Check: Confirm interview time (check timezone!), date, video links, interviewer names. Prepare your setup (quiet space, stable internet, water, charged devices).

- Relax & Rest: Avoid cramming. Trust your preparation. Get good sleep!

✅ **Output:** Clear and confident behavioral answers, strong understanding of key topics, practice under mock interview pressure, all logistics in place, and a calm, focused mindset.

📌 Understand ATS (Applicant Tracking System)

Most companies use an ATS (Applicant Tracking System) to scan resumes before a human sees them. If your resume doesn't have the right keywords, it may never reach the recruiter.

How to make your resume ATS-friendly:

- Use simple formatting (no tables or columns).
- Save it as a .docx or PDF (unless the company asks for something else).
- Include keywords from the job description (e.g., Kubernetes, CI/CD, Terraform).
- Avoid fancy designs, logos, or images.

✅ What to Include in Your DevOps Resume

- Title: "DevOps Engineer" or the role you're applying for.
- Summary: 2–3 lines about your experience and skills.
- Skills: Mention key DevOps tools (Docker, Jenkins, AWS, Kubernetes, etc.).
- Projects: Briefly describe projects with outcomes.
- Certifications: Add your CKA, AWS, or other DevOps-related certs.
- Experience: Focus on what you built, automated, or improved.
- Links: Add GitHub, LinkedIn, or portfolio if available.